

```

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>

// =====
// WIFI SETTINGS
// =====

const char* ssid = "Shah";
const char* password = "@15191304";

WebServer server(80);

// =====
// MPU6050 SETTINGS
// =====

#define MPU_ADDR 0x68

#define PWR_MGMT_1 0x6B
#define ACCEL_CONFIG 0x1C
#define GYRO_CONFIG 0x1B
#define CONFIG 0x1A
#define ACCEL_XOUT_H 0x3B

// =====
// BTS7960 PINS
// =====

#define RPWM_PIN 18
#define LPWM_PIN 19
#define REN_PIN 25
#define LEN_PIN 26

// =====
// PWM SETTINGS
// =====

// Keep speed low initially

```

```

#define MOTOR_PWM 80

#define PWM_FREQ 1000
#define PWM_RES 8

#define RPWM_CHANNEL 0
#define LPWM_CHANNEL 1

// =====
// MPU POSITION THRESHOLDS
// =====

// Based on your measured data:
//
// 0 degree -> AZ approximately 0.15
// 90 degree -> AZ approximately 1.03

#define ZERO_THRESHOLD 0.35
#define NINETY_THRESHOLD 1.04

// =====
// SAFETY SETTINGS
// =====

// Maximum time motor is allowed to run
#define MOTOR_TIMEOUT 15000

// =====
// SYSTEM STATE
// =====

enum MotorState {

    MOTOR_STOPPED,
    MOVING_TO_ZERO,
    MOVING_TO_NINETY

};

MotorState motorState = MOTOR_STOPPED;

```

```

unsigned long motorStartTime = 0;

// =====
// MPU VALUES
// =====

float ax = 0;
float ay = 0;
float az = 0;

float filteredAz = 0;

// =====
// FUNCTION: WRITE MPU REGISTER
// =====

void writeRegister(byte reg, byte data) {

    Wire.beginTransaction(MPU_ADDR);
    Wire.write(reg);
    Wire.write(data);
    Wire.endTransmission();

}

// =====
// FUNCTION: CHECK MPU CONNECTION
// =====

bool checkMPU() {

    Wire.beginTransaction(MPU_ADDR);

    byte error = Wire.endTransmission();

    if (error == 0) {
        return true;
    }
}

```

```

    return false;

}

// =====
// FUNCTION: READ MPU6050
// =====

bool readMPU() {

    Wire.beginTransaction(MPU_ADDR);
    Wire.write(ACCEL_XOUT_H);

    byte error = Wire.endTransmission(false);

    if (error != 0) {
        return false;
    }

    Wire.requestFrom(MPU_ADDR, 14, true);

    if (Wire.available() < 14) {
        return false;
    }

    int16_t rawAx = (Wire.read() << 8) | Wire.read();
    int16_t rawAy = (Wire.read() << 8) | Wire.read();
    int16_t rawAz = (Wire.read() << 8) | Wire.read();

    // Temperature - read but not used
    int16_t rawTemp = (Wire.read() << 8) | Wire.read();

    // Gyroscope - read but not used
    int16_t rawGx = (Wire.read() << 8) | Wire.read();
    int16_t rawGy = (Wire.read() << 8) | Wire.read();
    int16_t rawGz = (Wire.read() << 8) | Wire.read();

    // Convert to g

```

```

    ax = rawAx / 16384.0;
    ay = rawAy / 16384.0;
    az = rawAz / 16384.0;

    // Simple smoothing filter
    filteredAz = (filteredAz * 0.8) + (az * 0.2);

    return true;
}

// =====
// PWM SETUP
// Compatible with ESP32 Arduino Core 2.x and 3.x
// =====

void setupPWM() {

    #if ESP_ARDUINO_VERSION_MAJOR >= 3

        ledcAttach(RPWM_PIN, PWM_FREQ, PWM_RES);
        ledcAttach(LPWM_PIN, PWM_FREQ, PWM_RES);

    #else

        ledcSetup(RPWM_CHANNEL, PWM_FREQ, PWM_RES);
        ledcSetup(LPWM_CHANNEL, PWM_FREQ, PWM_RES);

        ledcAttachPin(RPWM_PIN, RPWM_CHANNEL);
        ledcAttachPin(LPWM_PIN, LPWM_CHANNEL);

    #endif

}

// =====
// FUNCTION: SET RPWM
// =====

```

```
void setRPWM(int value) {

    #if ESP_ARDUINO_VERSION_MAJOR >= 3

        ledcWrite(RPWM_PIN, value);

    #else

        ledcWrite(RPWM_CHANNEL, value);

    #endif

}

// =====
// FUNCTION: SET LPWM
// =====

void setLPWM(int value) {

    #if ESP_ARDUINO_VERSION_MAJOR >= 3

        ledcWrite(LPWM_PIN, value);

    #else

        ledcWrite(LPWM_CHANNEL, value);

    #endif

}

// =====
// MOTOR STOP
// =====

void stopMotor() {

    setRPWM(0);
    setLPWM(0);
}
```

```

    motorState = MOTOR_STOPPED;

    Serial.println("MOTOR STOPPED");

}

// =====
// MOVE TOWARDS 0 DEGREE
// =====

void moveToZeroMotor() {

    // Direction 1

    setRPWM(0);
    setLPWM(MOTOR_PWM);

}

// =====
// MOVE TOWARDS 90 DEGREE
// =====

void moveToNinetyMotor() {

    // Direction 2

    setRPWM(MOTOR_PWM);
    setLPWM(0);

}

// =====
// COMMAND: MOVE TO 0 DEGREE
// =====

void commandZero() {

    readMPU();

```

```

Serial.println();
Serial.println("COMMAND RECEIVED: MOVE TO 0 DEGREE");

Serial.print("Current AZ: ");
Serial.println(filteredAz, 3);

// Already at 0 degree

if (filteredAz <= ZERO_THRESHOLD) {

    Serial.println("Already at 0 DEGREE");

    stopMotor();

    return;

}

motorState = MOVING_TO_ZERO;

motorStartTime = millis();

moveToZeroMotor();

}

// =====
// COMMAND: MOVE TO 90 DEGREE
// =====

void commandNinety() {

    readMPU();

    Serial.println();
    Serial.println("COMMAND RECEIVED: MOVE TO 90 DEGREE");

    Serial.print("Current AZ: ");
    Serial.println(filteredAz, 3);

```

```

// Already at 90 degree

if (filteredAz >= NINETY_THRESHOLD) {

    Serial.println("Already at 90 DEGREE");

    stopMotor();

    return;

}

motorState = MOVING_TO_NINETY;

motorStartTime = millis();

moveToNinetyMotor();

}

// =====
// WEB PAGE
// =====

void handleRoot() {

    String stateText;

    if (motorState == MOTOR_STOPPED) {

        stateText = "STOPPED";

    }

    else if (motorState == MOVING_TO_ZERO) {

        stateText = "MOVING TO 0 DEGREE";

```

```
}

else if (motorState == MOVING_TO_NINETY) {

    stateText = "MOVING TO 90 DEGREE";

}

String page = R"rawliteral(

<!DOCTYPE html>

<html>

<head>

<meta name="viewport" content="width=device-width, initial-scale=1">

<title>ParkBnB Control</title>

<style>

body {

    font-family: Arial;
    text-align: center;

    background: #f4f4f4;

    margin-top: 50px;

}

h1 {

    color: #222;

}

.status {

    font-size: 20px;
```

```
margin: 20px;

padding: 15px;

background: white;

border-radius: 10px;

}

button {

width: 250px;

height: 80px;

font-size: 30px;

margin: 15px;

border: none;

border-radius: 15px;

cursor: pointer;

}

.zero {

background: #4CAF50;

color: white;

}

.ninety {

background: #f44336;

color: white;
```

```

}

.stop {

    background: #333;

    color: white;

}

</style>

</head>

<body>

<h1>ParkBnB Barrier Control</h1>

<div class="status">

STATUS: )rawliteral";

    page += stateText;

    page += R"rawliteral(

<br><br>

MPU AZ: )rawliteral";

    page += String(filteredAz, 3);

    page += R"rawliteral(

</div>

<a href="/zero">

<button class="zero">

0 DEGREE

```

```
</button>
```

```
</a>
```

```
<br>
```

```
<a href="/ninety">
```

```
<button class="ninety">
```

```
90 DEGREE
```

```
</button>
```

```
</a>
```

```
<br>
```

```
<a href="/stop">
```

```
<button class="stop">
```

```
STOP
```

```
</button>
```

```
</a>
```

```
</body>
```

```
</html>
```

```
)rawliteral";
```

```
server.send(200, "text/html", page);
```

```

}

// =====
// WEB COMMAND: 0 DEGREE
// =====

void handleZero() {

    commandZero();

    server.setHeader("Location", "/");

    server.send(303);

}

// =====
// WEB COMMAND: 90 DEGREE
// =====

void handleNinety() {

    commandNinety();

    server.setHeader("Location", "/");

    server.send(303);

}

// =====
// WEB COMMAND: STOP
// =====

void handleStop() {

    stopMotor();

    Serial.println("EMERGENCY STOP FROM WEB");

```

```

server.sendHeader("Location", "/");

server.send(303);

}

// =====
// SETUP
// =====

void setup() {

    Serial.begin(115200);

    // -----
    // I2C START
    // -----

    Wire.begin(21, 22);

    delay(1000);

    // -----
    // MPU CHECK
    // -----

    Serial.println();
    Serial.println("=====");
    Serial.println("          PARKBNB SYSTEM");
    Serial.println("=====");

    Serial.println("Checking MPU6050...");

    if (!checkMPU()) {

        Serial.println("ERROR: MPU6050 NOT FOUND!");

        while (true) {

```

```
        delay(1000);

    }

}

Serial.println("MPU6050 FOUND");

// Wake MPU6050

writeRegister(PWR_MGMT_1, 0x00);

delay(100);

// Accelerometer +-2g

writeRegister(ACCEL_CONFIG, 0x00);

// Gyroscope +-250 deg/sec

writeRegister(GYRO_CONFIG, 0x00);

// Low pass filter

writeRegister(CONFIG, 0x03);

Serial.println("MPU6050 INITIALIZED");

// -----
// MOTOR ENABLE PINS
// -----

pinMode(REN_PIN, OUTPUT);
pinMode(LEN_PIN, OUTPUT);
```

```
// Enable BTS7960

digitalWrite(REN_PIN, HIGH);
digitalWrite(LEN_PIN, HIGH);

// PWM SETUP

setupPWM();

stopMotor();

// -----
// WIFI CONNECT
// -----

Serial.println();
Serial.print("Connecting to WiFi: ");

Serial.println(ssid);

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

}

Serial.println();
Serial.println();

Serial.println("WIFI CONNECTED!");
```

```

Serial.print("OPEN THIS IP ADDRESS: ");

Serial.println(WiFi.localIP());

// -----
// WEB SERVER ROUTES
// -----

server.on("/", handleRoot);

server.on("/zero", handleZero);

server.on("/ninety", handleNinety);

server.on("/stop", handleStop);

server.begin();

Serial.println("WEB SERVER STARTED");
}

// =====
// MAIN LOOP
// =====

void loop() {

    // Handle web requests

    server.handleClient();

    // Read MPU continuously

    bool mpuOK = readMPU();

```

```

// If MPU fails while motor is moving,
// immediately stop for safety

if (!mpuOK && motorState != MOTOR_STOPPED) {

    Serial.println("MPU ERROR - MOTOR STOPPED FOR SAFETY");

    stopMotor();

    return;

}

// -----
// MOVING TO 0 DEGREE
// -----

if (motorState == MOVING_TO_ZERO) {

    if (filteredAz <= ZERO_THRESHOLD) {

        Serial.println("0 DEGREE REACHED");

        Serial.print("Final AZ: ");

        Serial.println(filteredAz, 3);

        stopMotor();

    }

}

// -----
// MOVING TO 90 DEGREE
// -----

if (motorState == MOVING_TO_NINETY) {

    if (filteredAz >= NINETY_THRESHOLD) {

```

```

        Serial.println("90 DEGREE REACHED");

        Serial.print("Final AZ: ");

        Serial.println(filteredAz, 3);

        stopMotor();

    }

}

// -----
// SAFETY TIMEOUT
// -----

if (motorState != MOTOR_STOPPED) {

    if (millis() - motorStartTime >= MOTOR_TIMEOUT) {

        Serial.println("SAFETY TIMEOUT!");

        Serial.println("MOTOR STOPPED");

        stopMotor();

    }

}

// Print MPU data occasionally

static unsigned long lastPrint = 0;

if (millis() - lastPrint >= 500) {

    lastPrint = millis();

```

```
Serial.print("AZ: ");

Serial.print(az, 3);

Serial.print(" | Filtered AZ: ");

Serial.print(filteredAz, 3);

Serial.print(" | State: ");

if (motorState == MOTOR_STOPPED) {

    Serial.println("STOPPED");

}

else if (motorState == MOVING_TO_ZERO) {

    Serial.println("MOVING TO 0");

}

else {

    Serial.println("MOVING TO 90");

}

}

delay(10);

}
```